

Huffman 编解码实验报告

宋亦寒 PB25000200

一、问题描述

1. 基本要求：一个完整的 Huffman 编解码系统应该具有以下功能：

- 初始化 (Initialization)。从终端读入字符集大小 n ，以及 n 个字符和 n 个权值，建立 Huffman 树，并将它存入 `hfmTree` 中。
- 编码 (Encoding)。利用已经建好的 Huffman 树（如果不在内存，则应从文件 `hfmTree` 中读取），对文件 `ToBeTran` 中的正文进行编码，然后将结果存入文件 `CodeFile` 中。
- 解码 (Decoding)。利用已经建立好的 Huffman 树将文件 `CodeFile` 中的代码进行解码，结果存入 `TextFile` 中。
- 打印代码文件 (Print)。将文件 `CodeFile` 以紧凑的格式显示在终端上，每行 50 个代码。同时将此字符形式的编码文件写入文件 `CodePrint` 中。
- 打印 Huffman 树 (Tree Printing)。将已经在内存中的 Huffman 树以直观的形式（树或者凹入的形式）显示在终端上，同时将此字符形式的 Huffman 树写入文件 `TreePrint` 中。

2. 测试数据：用下表给出的字符集和频度的实际统计数据建立 Huffman 树，并对以下报文进行编码和译码：“THIS PROGRAM IS MY FAVORITE”。

字符	频度	字符	频度	字符	频度
空格(SPACE)	186	A	64	B	13
C	22	D	32	E	103
F	21	G	15	H	47
I	57	J	1	K	5
L	32	M	20	N	57
O	63	P	15	Q	1
R	48	S	51	T	80
U	23	V	8	W	18
X	1	Y	16	Z	1

3. 输入输出：字符集大小 n ， n 个字符和 n 个权值均从终端读入，初始化后的 Huffman 树存储在 hfmTree 文件中，待编码文件为 ToBeTran，编码结果以文本的方式存储在文件 CodeFile 中，解码文件存在 TextFile 中，打印的编码和 Huffman 树分别存储在 CodePrint 和 TreePrint 文件中。用户界面可以设计为“菜单”方式：显示上述功能符号，再加入退出功能“Q”，表示退出（quit）。

二、需求分析

1. 本程序用来完成 Huffman 编/解码系统，能够建立 Huffman 树、保存 Huffman 树、对待传数据进行预先编码，并在接收端进行解码。
2. 输入形式：初始化时输入字符集大小 n ，以及 n 个字符和 n 个权值；编码时读取 ToBeTran 中的正文；解码时读取 CodeFile 中的代码；菜单输入功能符 I、E、D、P、T、Q。
3. 输入值范围： n 为正整数；每个字符对应一个正整数权值；待编码正文中的字符应属于已经建立的字符集。
4. 输出形式：初始化后的 Huffman 树存储在 hfmTree 文件中；编码结果以文本方式存储在 CodeFile 中；解码文件存在 TextFile 中；打印的编码和 Huffman 树分别存储在 CodePrint 和 TreePrint 文件中。
5. 测试数据要求：采用给出的字符集和频度，对测试报文进行编码和译码，并检查译码结果是否与原文一致。

三、程序设计

本节将实验报告所需内容中的“概要设计”和“详细设计”合并。

1. 所用到的数据结构及其 ADT

为了实现上述功能，采用顺序存储的 Huffman 树和顺序栈。Huffman 树结点保存权值、双亲、左右孩子、字符和编码；顺序栈保存从根到叶路径上的 0/1 编码。

ADT HuffmanTree{

数据对象： $D=\{HTNode[i] \mid i=1,2,\dots,2n-1\}$

数据关系： $R=\{\langle parent, lchild \rangle, \langle parent, rchild \rangle \mid parent, lchild, rchild \text{ 属于 } D\}$

基本操作：

CreateHuffmanTree(&data,w,c);

CreateHuffmanCoding(&data);

Encoding(&data);

Decoding(&data);

PrintCodeFile();

TreePrinting(&data);

```

} End ADT HuffmanTree
ADT SqStack{
    数据对象: D={elem[i] | i=0,1,...,top-1}
    基本操作:
        InitStack(&S);
        Push(&S,ch);
        Pop(&S,&ch);
} End ADT SqStack
    
```

2. 实现上述设计的数据结构

```

typedef struct{
    int weight;
    int parent, lchild, rchild;
    char ch;
    char code[1000];
} HTNode;
typedef struct{
    HTNode *tree;
    int n;
} HuffData;
typedef struct{
    char elem[1000];
    int top;
} SqStack;
    
```

3. 核心模块说明和对应算法伪码

Select	在尚未建立双亲的结点中选出权值最小的两个结点 s1 和 s2。
CreateHuffmanTree	申请 2n-1 个结点空间，按 Huffman 算法逐步合并最小权值结点。
CreateHuffmanCoding	从根结点出发，递归遍历 Huffman 树，为每个叶子字符生成 0/1 编码。
Initialization	读取字符和权值，建立 Huffman 树和编码，并将树写入 hfmTree.txt。
Encoding	读取 ToBeTran.txt，将正文按 Huffman 编码写入 CodeFile.txt。
Decoding	读取 CodeFile.txt，按 Huffman 树译码，将结果写入 TextFile.txt。
PrintCodeFile	将 CodeFile 每行 50 个代码显示，并写入 CodePrint.txt。
TreePrinting	将 Huffman 树输出到终端，并写入 TreePrint.txt。

CreateHuffmanTree(data,w,c)

$m = 2 * n - 1$

初始化 m 个结点

前 n 个结点存放字符和权值

后 $n-m$ 个结点:

Select(HT, i-1, s1, s2)

HT[i].lchild = s1; HT[i].rchild = s2

HT[i].weight = HT[s1].weight + HT[s2].weight

HT[s1].parent = HT[s2].parent = i

Coding(HT, root, S)

if 是叶子结点

将栈中从根到叶的 0/1 序列作为该字符编码

else

左分支入栈 0, 递归左子树, 退栈

右分支入栈 1, 递归右子树, 退栈

Encoding(data)

逐字符读取 ToBeTran

在叶子结点中查找该字符

将对应编码写入 CodeFile

Decoding(data)

从根结点开始逐位读取 CodeFile

读 0 走左孩子, 读 1 走右孩子

到达叶子时输出字符到 TextFile, 并回到根结点

4. 主程序流程及其模块调用关系

主函数模块

├ Initialization: 读入字符和权值, 建立 Huffman 树, 生成编码, 保存 hfmTree

| └ CreateHuffmanTree

| | └ Select

| └ CreateHuffmanCoding

| └ Coding

├ Encoding: 读取 ToBeTran 并写入 CodeFile

├ Decoding: 读取 CodeFile 并写入 TextFile

├ PrintCodeFile: 显示 CodeFile, 每行 50 个代码, 并写入 CodePrint

└ TreePrinting: 以凹入形式显示 Huffman 树, 并写入 TreePrint

初始化

显示菜单: I/E/D/P/T/Q

输入操作指令 choice

switch(choice)

```
I: Initialization(&data)
E: Encoding(&data)
D: Decoding(&data)
P: PrintCodeFile()
T: TreePrinting(&data)
Q: Quit
default: 输出“没有对应指令”
```

五、调试分析

1. 遇到的问题和解决方法

(1) 测试报文中含有空格字符，直接从文件按字符串读取字符集时不容易表示空格。解决方法：在初始化数据中用 SPACE 表示空格，读入后再转换成字符' '。

(2) 如果连续多次执行 Initialization, CreateHuffmanTree 会直接为 data->tree 分配新内存且没有释放上一次分配的内存，导致内存泄漏。解决方法：

```
// 如果树已经存在，应先释放内存

if (data->tree != NULL){

    free(data->tree);

    data->tree = NULL;

}

int m = n * 2 - 1;

data->tree = (HTNode *)malloc(sizeof(HTNode) * (m + 1));

...
```

(3) 编码和解码依赖已经建立好的 Huffman 树。若直接执行 E 或 D 而没有先执行 I，程序无法正确找到编码表或树根。解决方法：在 Encoding 和 Decoding 开始时检查 data->tree 是否为空。

2. 主要算法的时间和空间复杂度分析

- CreateHuffmanTree 每次合并都调用 Select 扫描已有结点，共进行 $n-1$ 次合并，时间复杂度为 $O(n^2)$ ，Huffman 树空间复杂度为 $O(n)$ 。
- CreateHuffmanCoding 需要遍历 Huffman 树，时间复杂度为 $O(n)$ ，递归栈和顺序栈额外空间复杂度为 $O(h)$ ， h 为树高。
- Encoding 设待编码正文长度为 L ，每个字符在线性扫描叶子结点中查找，时间复杂度为 $O(Ln)$ 。
- Decoding 设代码长度为 K ，每个代码位沿树走一步，时间复杂度为 $O(K)$ 。

六、使用说明

可运行指令：

Choice	含义
I	Initialization, 建立 Huffman 树并保存。
E	Encoding, 对 ToBeTran.txt 编码, 结果存入 CodeFile.txt。
D	Decoding, 对 CodeFile.txt 译码, 结果存入 TextFile.txt。
P	Print CodeFile, 终端显示编码, 每行 50 个代码, 同时写入 CodePrint.txt。
T	Tree Printing, 终端显示 Huffman 树, 同时写入 TreePrint.txt。
Q	Quit, 退出程序。

读取文件 init_input.txt 并初始化

读取测试报文: THIS PROGRAM IS MY FAVORITE

CodePrint.txt 内容如下, 每行 50 个代码:

```
11010001011000111111000100010100110000100101010110
01011101100011111110010100011111110011101011000001
001001001101101010
```

TextFile.txt 译码结果: THIS PROGRAM IS MY FAVORITE

TreePrint.txt 输出如下:

```

      186(' ')
    342
      80('T')
    156
      21('F')
    41
      20('M')
    76
      18('W')
    35
      5('K')
    9
      1('Z')
    2
      1('X')
```



```

                                4
                                1('Q')
                                2
                                1('J')
                                17
                                8('V')
592
                                32('L')
                                64
                                32('D')
128
                                64('A')
250
                                63('O')
122
                                16('Y')
                                31
                                15('P')
                                59
                                15('G')
                                28
                                13('B')
1000
                                57('N')
                                114
                                57('I')
217
                                103('E')
408
                                51('S')
                                99
                                48('R')
191
                                47('H')
                                92
                                23('U')
                                45
                                22('C')
```

其他初始化字符权重数据和报文也可以按照上述操作步骤读取、运行。

七、附录

源代码:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
/* Huffman 树结点: 保存权值、双亲、左右孩子、字符和编码。 */
typedef struct{
    int weight;
    int parent, lchild, rchild;
    char ch;
    char code[100];
} HTNode;
/* HuffData 保存 Huffman 树指针和字符集大小。 */
typedef struct{
    HTNode *tree;
    int n;
} HuffData;
/* SqStack 是生成编码时使用的顺序栈。 */
typedef struct{
    char elem[100];
    int top;
} SqStack;
/* 顺序栈用于记录从根到当前结点路径上的 0/1 编码。 */
void InitStack(SqStack *S){
    S->top = 0;
}
void Push(SqStack *S, char ch){
    S->elem[S->top++] = ch;
}
void Pop(SqStack *S, char *ch){
    if (S->top > 0){
        *ch = S->elem[--S->top];
    }
}
/* 在未设置双亲的结点中选出两个最小权值结点。 */
void Select(HTNode *HT, int end, int *s1, int *s2){
    int min1 = 0, min2 = 0;
    int m1 = 0, m2 = 0;
    int i;
    for (i = 1; i <= end; i++){
        if (HT[i].parent == 0){
```



```
        if (min1 == 0){
            min1 = i;
            m1 = HT[i].weight;
        } else if (min2 == 0){
            min2 = i;
            m2 = HT[i].weight;
            break;
        }
    }
}

if (m1 > m2){
    int tempWeight = m1; m1 = m2; m2 = tempWeight;
    int tempIndex = min1; min1 = min2; min2 = tempIndex;
}

for (i = i + 1; i <= end; i++){
    if (HT[i].parent == 0){
        if (HT[i].weight < m1){
            m2 = m1;
            min2 = min1;
            m1 = HT[i].weight;
            min1 = i;
        } else if (HT[i].weight < m2){
            m2 = HT[i].weight;
            min2 = i;
        }
    }
}

*s1 = min1;
*s2 = min2;
}

/* 根据字符权值构造 Huffman 树，数组从下标 1 开始使用。 */
void CreateHuffmanTree(HuffData *data, int *w, char *c){
    int n = data->n;
    if (n <= 1) return;
    int m = n * 2 - 1;
    data->tree = (HTNode *)malloc(sizeof(HTNode) * (m + 1));
    for (int i = 1; i <= m; i++){
        data->tree[i].weight = (i <= n) ? w[i - 1] : 0;
        data->tree[i].ch = (i <= n) ? c[i - 1] : '\0';
        data->tree[i].lchild = data->tree[i].rchild = data->tree[i].parent = 0;
        /* 初始时双亲和孩子均置 0，表示结点尚未连接。 */
        data->tree[i].code[0] = '\0';
    }
}
```

```
for (int i = n + 1; i <= m; i++){
    int s1, s2;
    Select(data->tree, i - 1, &s1, &s2);
    /* 每轮合并两个权值最小的根结点。 */
    data->tree[i].lchild = s1;
    data->tree[i].rchild = s2;
    data->tree[i].weight = data->tree[s1].weight + data->tree[s2].weight;
    data->tree[s1].parent = data->tree[s2].parent = i;
}
}

/* 递归生成编码：左分支记 0，右分支记 1，叶子结点得到字符编码。 */
void Coding(HTNode *HT, int root, SqStack *S){
    if (root != 0){
        if (HT[root].lchild == 0){
            Push(S, '\0');
            /* 压入字符串结束符，便于复制当前路径为编码串。 */
            strcpy(HT[root].code, S->elem);
            char ch;
            Pop(S, &ch);
        } else{
            Push(S, '0');
            /* 左分支入栈 0，递归返回后弹出。 */
            Coding(HT, HT[root].lchild, S);
            char ch;
            Pop(S, &ch);
            Push(S, '1');
            /* 右分支入栈 1，递归返回后弹出。 */
            Coding(HT, HT[root].rchild, S);
            Pop(S, &ch);
        }
    }
}

void CreateHuffmanCoding(HuffData *data){
    SqStack S;
    InitStack(&S);
    Coding(data->tree, 2 * data->n - 1, &S);
}

/* 初始化：读取字符集和权值，建立 Huffman 树并生成编码。 */
void Initialization(HuffData *data){
    FILE *fin = fopen("init_input.txt", "r");
    if (!fin){
        printf("未找到 init_input.txt.\n");
        return;
    }
}
```

```
}
fscanf(fin, "%d", &data->n);
int *w = (int *)malloc(sizeof(int) * (data->n));
char *c = (char *)malloc(sizeof(char) * (data->n));
char str[20];
int weight;
for (int i = 0; i < data->n; i++){
    fscanf(fin, "%s %d", str, &weight);
    if (strcmp(str, "SPACE") == 0){
        /*读入 SPACE 后转换为真正的空格字符。*/
        c[i] = ' ';
    } else{
        c[i] = str[0];
    }
    w[i] = weight;
}
fclose(fin);
CreateHuffmanTree(data, w, c);
CreateHuffmanCoding(data);
FILE *fout = fopen("hfmTree.txt", "wb");
int m = 2 * data->n - 1;
fwrite(&data->n, sizeof(int), 1, fout);
fwrite(data->tree + 1, sizeof(HTNode), m, fout);
/* tree[0]未使用, 因此从 tree+1 开始保存有效结点。*/
fclose(fout);
FILE *outText = fopen("ToBeTran.txt", "w");
fprintf(outText, "THIS PROGRAM IS MY FAVORITE");
fclose(outText);
free(w);
free(c);
printf("初始化完成\n");
}

/* 编码: 读取 ToBeTran.txt, 将正文转换为 01 编码写入 CodeFile.txt。*/
void Encoding(HuffData *data){
    if (!data->tree){
        printf("请先初始化\n");
        return;
    }
    FILE *fin = fopen("ToBeTran.txt", "r");
    FILE *fout = fopen("CodeFile.txt", "w");
    if (!fin || !fout){
        printf("打开文件失败\n");
        return;
    }
}
```

```
}
int ch;
while ((ch = fgetc(fin)) != EOF){
    int idx = -1;
    for (int i = 1; i <= data->n; i++){
        if (data->tree[i].ch == (char)ch && data->tree[i].lchild == 0 && data->tree[i].rchild == 0){
            idx = i;
            break;
        }
    }
    if (idx != -1){
        fprintf(fout, "%s", data->tree[idx].code);
    }
}
fclose(fin);
fclose(fout);
printf("编码成功 (CodeFile.txt)\n");
}

/* 解码: 按 01 序列沿 Huffman 树向下走, 到叶子结点输出字符。 */
void Decoding(HuffData *data){
    if (!data->tree){
        printf("请先初始化\n");
        return;
    }
    FILE *fin = fopen("CodeFile.txt", "r");
    FILE *fout = fopen("TextFile.txt", "w");
    if (!fin || !fout){
        printf("打开文件失败\n");
        return;
    }
    int root = 2 * data->n - 1;
    int curr = root;
    int bit;
    while ((bit = fgetc(fin)) != EOF){
        if (bit == '0'){
            curr = data->tree[curr].lchild;
            /* 读到 0 走左孩子。 */
        } else if (bit == '1'){
            curr = data->tree[curr].rchild;
            /* 读到 1 走右孩子。 */
        }
    }
    if (data->tree[curr].lchild == 0 && data->tree[curr].rchild == 0){
        fputc(data->tree[curr].ch, fout);
    }
}
```

```
        curr = root;
        /* 输出一个字符后回到根结点继续译码。 */
    }
}
fclose(fin);
fclose(fout);
printf("解码完成 (TextFile.txt)\n");
}
/* 打印编码文件：每行 50 个代码，并写入 CodePrint.txt。 */
void PrintCodeFile(){
    FILE *fin = fopen("CodeFile.txt", "r");
    FILE *fout = fopen("CodePrint.txt", "w");
    if (!fin || !fout){
        printf("Error opening files. Have you encoded the text?\n");
        return;
    }
    int ch;
    int count = 0;
    printf("CodeFile.txt Content:\n");
    while ((ch = fgetc(fin)) != EOF){
        putchar(ch);
        fputc(ch, fout);
        count++;
        if (count % 50 == 0){
            /* 实验要求编码文件每行显示 50 个代码。 */
            putchar('\n');
            fputc('\n', fout);
        }
    }
    if (count % 50 != 0){
        putchar('\n');
        fputc('\n', fout);
    }
    fclose(fin);
    fclose(fout);
    printf("打印编码完成 (CodePrint.txt)\n");
}
/* 后序遍历输出 Huffman 树。 */
void PrintNode(HTNode *HT, int root, int depth, FILE *fout){
    if (root == 0) return;
    PrintNode(HT, HT[root].rchild, depth + 1, fout);
    for (int i = 0; i < depth; i++){
        printf("    ");
    }
```

```
        fprintf(fout, "    ");
    }
    if (HT[root].lchild == 0 && HT[root].rchild == 0){
        printf("%d('%c')\n", HT[root].weight, HT[root].ch);
        fprintf(fout, "%d('%c')\n", HT[root].weight, HT[root].ch);
    } else{
        printf("%d\n", HT[root].weight);
        fprintf(fout, "%d\n", HT[root].weight);
    }
    PrintNode(HT, HT[root].lchild, depth + 1, fout);
}

void TreePrinting(HuffData *data){
    if (!data->tree){
        printf("请先初始化\n");
        return;
    }
    FILE *fout = fopen("TreePrint.txt", "w");
    PrintNode(data->tree, 2 * data->n - 1, 0, fout);
    /* Huffman 树根结点下标为 2n-1。 */
    fclose(fout);
    printf("HuffmanTree 打印完成 (TreePrint.txt)\n");
}

/* 主菜单：调用初始化、编码、解码和打印功能。 */
int main(){
    HuffData data;
    data.tree = NULL;
    data.n = 0;
    char choice[10];
    do{
        printf(" I. Initialization\n");
        printf(" E. Encoding\n");
        printf(" D. Decoding\n");
        printf(" P. Print CodeFile\n");
        printf(" T. Tree Printing\n");
        printf(" Q. Quit\n\n");
        printf("输入操作指令: ");
        scanf("%s", choice);
        switch (choice[0]){
            case 'I': case 'i': Initialization(&data); break;
            case 'E': case 'e': Encoding(&data); break;
            case 'D': case 'd': Decoding(&data); break;
            case 'P': case 'p': PrintCodeFile(); break;
            case 'T': case 't': TreePrinting(&data); break;
        }
    } while (choice[0] != 'Q' && choice[0] != 'q');
```



```
        case 'Q': case 'q': break;
        default: printf("没有对应指令\n"); break;
    }
} while (choice[0] != 'Q' && choice[0] != 'q');
if (data.tree){
    free(data.tree);
}
return 0;
}
```